SUNHAVEN CARE EMPLOYEE ACCESS SYSTEM

Easy Project Guide for a Non-Technical Person

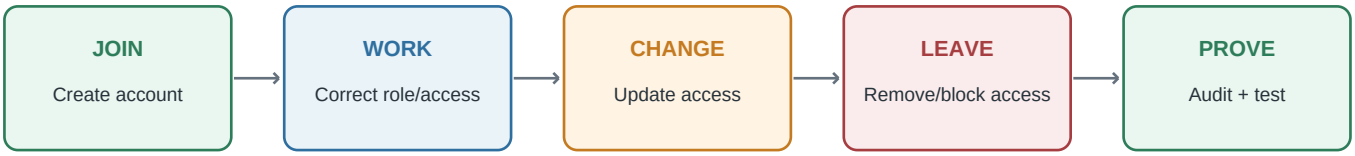
In one sentence

We are building a small system that gives Sunhaven Care employees the correct system access when they join, changes their access when their job changes, and removes their access when they leave.

1. What is the project?

Sunhaven Care has different workers such as Nurses, Care Workers, Managers and Agency Workers. These people should not all see the same information. The project controls **who can log in and what they can access**.

It is **not** a payroll system, salary system, roster system or full HR system.



Example

Sarah joins as a Nurse. The system creates her employee record, gives Nurse access and allows only the information she needs. Later Sarah becomes a Manager, so the old Nurse access is changed. When Sarah leaves Sunhaven, her account is disabled and she can no longer enter the protected system.

Main goal

Right person + right access + right time. When the person no longer needs access, remove it quickly and keep evidence showing what happened.

2. Main user roles

Role	Easy meaning
CareWorker	Can see only the care information assigned to them.
Nurse	Can access nursing/clinical information allowed for Nurses.
Manager	Can manage/review wider staff access.
AgencyWorker	Temporary access with sponsor and end date.
Auditor	Can review logs, evidence and access-review information.
IAMOperator	Admin role used to manage employee identities and access.

3. Features we need to build

The system can stay small. These are the important features.

Feature	What it does in simple words
Login	Employee signs in securely using Microsoft Entra ID. MFA can ask for a second verification step.
Dashboard	Shows basic counts such as active workers, leavers, agency workers, pending reviews and recent events.
Employees	List all test employees and show role, status, facility, manager/sponsor and important dates.
Joiner	Add a new worker, check the details, create/reconcile the identity and give the correct role.
Mover	When a worker changes job, remove old access first and then give the new approved access.
Leaver	Disable the account, remove roles/access, remove assignments and block portal access.
Residents	Show only fictional resident information and only to workers who are allowed to see it.
Protected pages	Pages such as Clinical, Care Notes or Review should open only for the correct role.
Access Review	Manager/Auditor chooses Retain, Change, Remove or Temporary Exception for a worker's access.
Audit Log	Records who did what, when, to whom and whether it worked.
Compliance Checker	Checks simple rules and shows PASS/FAIL when something is wrong.
403 Access Denied	If a person tries to open something they are not allowed to see, the system denies access and records it.

Important data rule

Use fictional employees and fictional residents only. Do not use real healthcare or real employee data.

4. What programming and software should we use?

Keep the technology simple so the project is easy to build, explain and demonstrate.

Part	Use	Easy explanation
Frontend	HTML + CSS + Bootstrap + small JavaScript	What the user sees: forms, buttons, tables and dashboard.
Backend	Python + Flask	The main program that receives input, applies rules and controls access.
Database	SQLite	Stores employee records, assignments, reviews and logs. No separate databa
Login / MFA	Microsoft Entra ID	Handles secure employee login and multi-factor authentication.
Microsoft connection	Microsoft Graph API from Python	Lets Python create/update/disable identities and manage access.
Reports	Python -> CSV / JSON / HTML	Creates evidence and test output that can be shown to the teacher.
Code storage	GitHub	Stores the code and provides evidence of project work.

Do we need PowerShell?

No. The recommended version uses Python to call Microsoft Graph API. This keeps the project easier to understand and reduces the number of technologies.

5. How the finished system works

- **Step 1:** Admin/manager opens the Sunhaven web system.
- **Step 2:** Employee information is entered or selected.
- **Step 3:** Python checks whether the information is valid.
- **Step 4:** SQLite stores the local workforce record.
- **Step 5:** Python uses Microsoft Graph API when Entra ID must be created, changed or disabled.
- **Step 6:** The worker receives only the role/access they need.
- **Step 7:** Every important action is written to the audit log.
- **Step 8:** Tests confirm the final state is correct.

Simple technical flow

User -> Web pages -> Python -> SQLite + Microsoft Graph -> Microsoft Entra ID -> Result + Audit Evidence

6. How do we test the project?

Testing means we try important situations and check that the system gives the expected result. The final evidence should come from the real running system, not from a mock-up image.

Test	What we do	Expected result
1. Joiner	Add a valid Nurse	Nurse is created and correct access works
2. Invalid employee	Use missing/duplicate information	System rejects it safely
3. Agency worker	Leave sponsor/end date empty	System rejects it
4. Correct access	Nurse opens an allowed page	Access allowed
5. Wrong access	CareWorker opens Manager-only page	Access denied / 403
6. Mover	Change Nurse to Manager	Old access removed, Manager access added
7. Leaver	Mark worker as left	Account/access disabled or removed
8. Leaver login	Try to sign in after leaving	Denied
9. Old session	Use an existing session after leaver action	Next protected request denied
10. Logout	Sign out	Session ends and protected pages require login
11. Audit	Perform a lifecycle action	Correct event appears with no secrets
12. Access review	Manager selects Retain/Change/Remove	Decision saved
13. Compliance	Create a deliberate bad condition	Checker shows FAIL
14. Fix problem	Correct the problem and run checker again	Finding clears / PASS

What evidence should we keep?

- Real screenshots from the running project.
- GitHub code/commit evidence.
- Test PASS/FAIL results.
- Audit log output.
- Before-and-after evidence for Mover and Leaver.
- Short notes explaining what was tested and what happened.

7. Final project requirement checklist

Use this table before submission. Tick the box only when the feature is actually working and you have evidence.

Check	Requirement	What proves it
[ ]	Employee records can be added/viewed/updated	Running employee pages + database evidence
[ ]	Roles exist: CareWorker, Nurse, Manager, AgencyWorker, Auditor, IAMOperator	Role list + working role tests
[ ]	Secure login works	Successful Entra login screenshot
[ ]	MFA is demonstrated where available	MFA prompt / tenant evidence
[ ]	Server-side RBAC protects pages	Allowed test + 403 denied test
[ ]	Joiner process works	New worker created with correct access
[ ]	Mover process works	Old access removed and new access added
[ ]	Leaver process works	Identity/access disabled/removed
[ ]	Leaver cannot access protected system	New login and/or next protected request denied
[ ]	Agency worker requires sponsor and end date	Negative validation test
[ ]	Resident access uses fictional data only	Synthetic resident page
[ ]	Resident access is limited by role/assignment	Allowed + denied tests
[ ]	Access Review page works	Saved manager/auditor decision
[ ]	Audit log works	Timestamped audit events
[ ]	Audit output does not expose passwords/tokens/secrets	Reviewed sanitised log
[ ]	Compliance checker finds seeded problems	PASS/FAIL findings page/report
[ ]	Compliance remediation can be re-tested	Finding clears after approved fix
[ ]	GitHub contains final source code	Repository evidence
[ ]	README/setup instructions exist	README file
[ ]	Architecture/design explanation exists	Diagram/document
[ ]	Testing evidence is complete	Test table + screenshots/logs
[ ]	Only synthetic/test data is used	Data review
[ ]	Final report explains design changes and limitations	Assessment 3 report
[ ]	Demo video shows working prototype and is under 10 minutes	Final video

When is the project complete?

The project is complete when the core employee lifecycle works end to end: a worker can join, receive the correct access, change role safely, leave, lose access, and the team can prove each important step using real tests and audit evidence.

8. Final demo - easiest order

The final demonstration should be simple and tell one clear story from start to finish.

- Explain the problem in about 30 seconds.
- Show login and dashboard.
- Add one Joiner.
- Change that employee's role (Mover).
- Show one allowed page and one 403 denied page.
- Process the employee as a Leaver and prove access is denied.
- Show Access Review.
- Run Compliance Checker.
- Show Audit Log and test results.
- End with limitations and what would be improved in a real production system.

Very simple final story

Join -> give correct access -> change role -> remove old access -> leave -> block access -> prove everything with logs and tests.

What should the non-technical person remember?

- This is an **employee access-control project**, not a full HR system.
- The most important feature is the **Joiner-Mover-Leaver** process.
- The most important security rule is **least privilege**: give only the access needed.
- The most important final proof is that a **Leaver can no longer access the system**.
- Keep the build simple: **Python + Flask + SQLite + HTML/CSS + Entra ID + Graph API**.
- Every important feature should have a real test and real evidence.

Source basis

Prepared from the supplied COIT13236 Assessment 1/2/3 specifications, the Sunhaven Care revised project proposal, the Sunhaven Care Security Policy and Procedure Set, the Access Management Policy, and the team's existing implementation/evidence files.